My Project

Generated by Doxygen 1.13.2

1 Hierarchical Index 1

1.1 Class Hierarchy . . . 1

2 Class Index 3

2.1 Class List . . . 3

3 File Index 5

3.1 File List . . . 5

4 Class Documentation 7

4.1 Automata Class Reference . . . 7

4.2 CFG< V_t, S_t > Class Template Reference . . . 7

4.3 DFA< Q_t, S_t > Class Template Reference . . . 8

4.4 epsilon Class Reference . . . 9

4.5 std::hash< std::list< T > > Struct Template Reference . . . 9

4.6 std::hash< std::pair< S, T > > Struct Template Reference . . . 9

4.7 std::hash< std::set< S > > Struct Template Reference . . . 9

4.8 std::hash< std::unordered_map< S, T, H > > Struct Template Reference . . . 10

4.9 std::hash< std::unordered_set< T > > Struct Template Reference . . . 10

4.10 std::hash< std::vector< T > > Struct Template Reference . . . 10

4.11 NFA< Q_t, S_t > Class Template Reference . . . 10

4.11.1 Detailed Description . . . 11

4.12 Parse_Tree Class Reference . . . 12

4.13 Parse_Tree_Node Class Reference . . . 12

4.14 PDA Class Reference . . . 12

4.15 RegEx Class Reference . . . 13

4.16 CFG< V_t, S_t >::Symbols Class Reference . . . 13

5 File Documentation 15

5.1 Automata.h . . . 15

5.2 CFG.h . . . 15

5.3 DFA.h . . . 16

5.4 include/NFA.h File Reference . . . 19

5.4.1 Detailed Description . . . 19

5.5 NFA.h . . . 20

5.6 PDA.h . . . 24

5.7 Regex.h . . . 24

5.8 Regular_Language.h . . . 26

5.9 STL_util.h . . . 27

5.10 STL_util.h . . . 35

Index 45

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Automata . . . 7
CFG< V_t, S_t > . . . 7
    PDA . . . 12
DFA< Q_t, S_t > . . . 8
std::hash< std::list< T > > . . . 9
std::hash< std::pair< S, T > > . . . 9
std::hash< std::set< S > > . . . 9
std::hash< std::unordered_map< S, T, H > > . . . 10
std::hash< std::unordered_set< T > > . . . 10
std::hash< std::vector< T > > . . . 10
NFA< Q_t, S_t > . . . 10
Parse_Tree . . . 12
Parse_Tree_Node . . . 12
RegEx . . . 13
S_t
    epsilon . . . 9
T_t
    CFG< V_t, S_t >::Symbols . . . 13
V_t
    CFG< V_t, S_t >::Symbols . . . 13

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Automata . . . 7
CFG< V_t, S_t > . . . 7
DFA< Q_t, S_t > . . . 8
epsilon . . . 9
std::hash< std::list< T > > . . . 9
std::hash< std::pair< S, T > > . . . 9
std::hash< std::set< S > > . . . 9
std::hash< std::unordered_map< S, T, H > > . . . 10
std::hash< std::unordered_set< T > > . . . 10
std::hash< std::vector< T > > . . . 10
NFA< Q_t, S_t >
    Q_t is the type for the for states and the S_t is the type for transitions . . . 10
Parse_Tree . . . 12
Parse_Tree_Node . . . 12
PDA . . . 12
RegEx . . . 13
CFG< V_t, S_t >::Symbols . . . 13

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

include/Automata.h 15
include/CFG.h 15
include/DFA.h 16
include/NFA.h
NFA implementation 19
include/PDA.h 24
include/Regex.h 24
include/Regular_Language.h 26
include/STL_util.h 27
include/test/STL_util.h 35

Chapter 4

Class Documentation

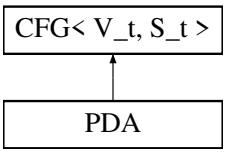
4.1 Automata Class Reference

The documentation for this class was generated from the following file:

- include/Automata.h

4.2 CFG< V_t, S_t > Class Template Reference

Inheritance diagram for CFG< V_t, S_t >:



Classes

- class Symbols

Public Member Functions

- Setv **Epsilon_Productions** () const

Public Attributes

- Setv **V_**
- unordered_set< T_t > **sigma_**
- unordered_map< V_t, Bodies > **P_**
- V_t **S_**
- Setv **nullables_**

Private Types

- using **Body** = vectors<Symbols>
- using **Bodies** = unordered_set<Body>
- using **Setv** = unordered_set<V_t>

The documentation for this class was generated from the following file:

- include/CFG.h

4.3 DFA< Q_t, S_t > Class Template Reference

Public Member Functions

- **DFA** (const SetUs &sigma, const SetUq &Q, const SetUq &F, const Q_t &q0, const Mapt &delta)
- **DFA** (const DFA &M)
- bool **operator()** (const vector< S_t > &word, bool verbose=false)
- SuperSetUq **Moore_partition_** (const SuperSetUq &P) const
- SuperSetUq **Moore_partition** (const SetUq &S, const SuperSetUq &P)
- DFA< SetUq, S_t > **minimizeDFA** () const
- DFA< PairQQ, S_t > **intersection_** (const DFA &M) const
- DFA< PairQQ, S_t > **union_** (const DFA &M) const
- DFA & **compDFA** ()
- DFA< int, S_t > **baseDFA** () const

Public Attributes

- Q_t **q0_**
- SetUq **Q_**
- SetUs **sigma_**
- Mapt **delta_**
- SetUq **F_**

Private Types

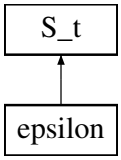
- using **SetUq** = unordered_set<Q_t>
- using **SetUs** = unordered_set<S_t>
- using **SuperSetUq** = unordered_set< SetUq >
- using **PairQS** = pair<Q_t, S_t>
- using **PairQQ** = pair<Q_t, Q_t>
- using **Mapt** = unordered_map<PairQS, Q_t>

The documentation for this class was generated from the following files:

- include/DFA.h
- src/DFA.cpp

4.4 epsilon Class Reference

Inheritance diagram for epsilon:



The documentation for this class was generated from the following file:

- src/Regular_Language.cpp

4.5 std::hash< std::list< T > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::list< T > &x) const
- size_t **operator()** (const std::list< T > &x) const

The documentation for this struct was generated from the following files:

- include/STL_util.h
- include/test/STL_util.h

4.6 std::hash< std::pair< S, T > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::pair< S, T > &x) const
- size_t **operator()** (const std::pair< S, T > &x) const

The documentation for this struct was generated from the following files:

- include/STL_util.h
- include/test/STL_util.h

4.7 std::hash< std::set< S > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::set< S > &x) const
- size_t **operator()** (const std::set< S > &x) const

The documentation for this struct was generated from the following files:

- include/STL_util.h
- include/test/STL_util.h

4.8 std::hash< std::unordered_map< S, T, H > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::unordered_map< S, T, H > &x) const
- size_t **operator()** (const std::unordered_map< S, T, H > &x) const

The documentation for this struct was generated from the following files:

- include/STL_util.h
- include/test/STL_util.h

4.9 std::hash< std::unordered_set< T > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::unordered_set< T > &x) const
- size_t **operator()** (const std::unordered_set< T > &x) const

The documentation for this struct was generated from the following files:

- include/STL_util.h
- include/test/STL_util.h

4.10 std::hash< std::vector< T > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::vector< T > &x) const
- size_t **operator()** (const std::vector< T > &x) const

The documentation for this struct was generated from the following files:

- include/STL_util.h
- include/test/STL_util.h

4.11 NFA< Q_t, S_t > Class Template Reference

Q_t is the type for the for states and the S_t is the type for transitions.

`#include <NFA.h>`

Public Member Functions

- **NFA** (const SetUs &sigma, const SetUq &Q, const SetUq &F, const Q_t &q0, const Mapt &delta, const Map &epDt)
- **NFA** (const NFA &N)
- NFA **operator=** (const NFA &N)
- bool **operator()** (const vector< S_t > &word, bool verbose=false)
- NFA< int, S_t > **union_** (const NFA &N) const
- NFA< int, S_t > **concatenate** (const NFA &N) const
- NFA< int, S_t > **KleneStar** () const
- NFA< int, S_t > **baseNFA** (int start=0) const
- SetUq **epsilon_closure** (const SetUq &S) const
- void **insert** (const S_t &a)
- int **size** () const

Public Attributes

- Q_t **q0_**
- SetUq **Q_**
- SetUs **sigma_**
- Mapt **delta_**
- SetUq **F_**
- Map **epDt_**

Private Types

- using **SetUq** = unordered_set<Q_t>

  i cant use a typedef for templates so i am using "using"
- using **SetUs** = unordered_set<S_t>
- using **SuperSetUq** = unordered_set< SetUq >
- using **PairQS** = pair<Q_t, S_t>
- using **PairQQ** = pair<Q_t, Q_t>
- using **PairS** = pair<int, S_t>
- using **Mapt** = unordered_map<PairQS, SetUq>
- using **SetUi** = unordered_set<int>
- using **Mapi** = unordered_map<PairS, SetUi>
- using **Map** = unordered_map<Q_t, SetUq>
- using **MapUi** = unordered_map<int, SetUi>

4.11.1 Detailed Description

template<typename Q_t, typename S_t>
class NFA< Q_t, S_t >

Q_t is the type for the for states and the S_t is the type for transitions.

The documentation for this class was generated from the following file:

- include/NFA.h

4.12 Parse_Tree Class Reference

Private Attributes

- Parse_Tree_Node **root_**

The documentation for this class was generated from the following file:

- include/CFG.h

4.13 Parse_Tree_Node Class Reference

Private Attributes

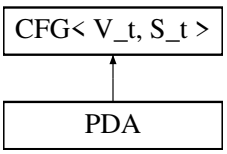
- Token **x**
- vector< Parse_Tree_Node > **children_**

The documentation for this class was generated from the following file:

- include/CFG.h

4.14 PDA Class Reference

Inheritance diagram for PDA:



Private Member Functions

- bool **operator()** (const std::vector< S_t > &)
- **CYK** ()
- **SRP** ()

Additional Inherited Members

Public Member Functions inherited from CFG< V_t, S_t >

- Setv **Epsilon_Productions** () const

Public Attributes inherited from CFG< V_t, S_t >

- Setv **V_**
- unordered_set< T_t > **sigma_**
- unordered_map< V_t, Bodies > **P_**
- V_t **S_**
- Setv **nullables_**

The documentation for this class was generated from the following file:

- include/PDA.h

4.15 RegEx Class Reference

Public Member Functions

- **RegEx** (char c)
- **RegEx** (const string &x)
- **RegEx** (const NFA< int, char > &N)
- **RegEx** (const RegEx &r)
- RegEx & **operator=** (const RegEx &r)
- RegEx & **operator|=** (const RegEx &r)
- const RegEx **operator|** (const RegEx &r) const
- RegEx **operator+** () const
- RegEx **operator*** () const
- RegEx & **operator&=** (const RegEx &r)
- RegEx **operator&** (const RegEx &r) const
- DFAic **powerset_construction** () const

Public Attributes

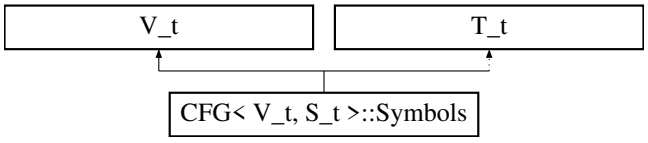
- NFA< int, char > **N_**
- string **regex_**

The documentation for this class was generated from the following file:

- include/Regex.h

4.16 CFG< V_t, S_t >::Symbols Class Reference

Inheritance diagram for CFG< V_t, S_t >::Symbols:



Public Member Functions

- **Symbols** (const V_t &v)
- **Symbols** (const T_t &t)

Public Attributes

- const bool **is_terminal**

The documentation for this class was generated from the following file:

- include/CFG.h

Chapter 5

File Documentation

5.1 Automata.h

```
00001 #ifndef AUTOMATA_H
00002 #define AUTOMATA_H
00003
00004 #include "DFA.h"
00005 #include "NFA.h"
00006 #include "Regex.h"
00007 #include "CFG.h"
00008 #include "PDA.h"
00009
00010 class Automata
00011 {
00012 public:
00013 private:
00014 };
00015
00016 #endif
```

5.2 CFG.h

```
00001 #ifndef CFG_H
00002 #define CFG_H
00003
00004 template <typename V_t, typename S_t >
00005 class CFG
00006 {
00007     class Symbols: public V_t, T_t
00008     {
00009     public:
00010         Symbols(const V_t & v)
00011             : V_t(v), is_terminal(false)
00012         {}
00013         Symbols(const T_t & t)
00014             : T_t(t), is_terminal(true)
00015         {}
00016         const bool is_terminal;
00017
00018     };
00019     using Body   = vectors<Symbols>;
00020     using Bodies = unordered_set<Body>;
00021     using Setv   = unordered_set<V_t>;
00022 public:
00023
00024     Setv Epsilon_Productions() const;
00025
00026     Setv V_;
00027     unordered_set<T_t> sigma_;
00028     unordered_map<V_t, Bodies> P_;
00029     V_t S_;
00030     Setv nullables_;
00031 };
00032
00033 class Parse_Tree
00034 {
```

```
00035     Parse_Tree_Node root_;
00036 };
00037
00038 class Parse_Tree_Node
00039 {
00040     Token x;
00041     vector<Parse_Tree_Node> children_;
00042 };
00043
00044 #endif
```

5.3 DFA.h

```
00001 #ifndef DFA_H
00002 #define DFA_H
00003
00004 #include "STL_util.h"
00005
00006 template < typename Q_t, typename S_t >
00007 class DFA;
00008
00009 template < typename Q_t, typename S_t >
00010 void walk(const pair<Q_t, Q_t> & qi,
00011           const unordered_map< pair<Q_t, S_t>, Q_t> & delta0,
00012           const unordered_map< pair<Q_t, S_t>, Q_t> & delta1,
00013           unordered_map< pair< pair<Q_t, Q_t> , S_t>,
00014           pair<Q_t, Q_t> > & delta_, const unordered_set<S_t> & alphabet,
00015           unordered_set< pair<Q_t, Q_t> > & states);
00016
00017 template <typename Q_t, typename S_t>
00018 class DFA
00019 {
00020     using SetUq = unordered_set<Q_t>;
00021     using SetUs = unordered_set<S_t>;
00022     using SuperSetUq = unordered_set< SetUq >;
00023     using PairQS = pair<Q_t, S_t>;
00024     using PairQQ = pair<Q_t, Q_t>;
00025     using Mapt = unordered_map<PairQS, Q_t>;
00026 public:
00027     DFA(const SetUs & sigma, const SetUq & Q,
00028         const SetUq & F,     const Q_t   & q0,
00029         const Mapt  & delta);
00030     DFA(const DFA & M);
00031     bool operator()(const vector<S_t> & word, bool verbose=false);
00032     SuperSetUq Moore_partition_(const SuperSetUq & P) const;
00033     SuperSetUq Moore_partition(const SetUq & S, const SuperSetUq & P);
00034     DFA<SetUq, S_t> minimizeDFA() const;
00035     DFA<PairQQ, S_t> intersection_(const DFA & M) const;
00036     DFA<PairQQ, S_t> union_(       const DFA & M) const;
00037     DFA & compDFA();
00038     DFA<int, S_t> baseDFA() const;
00039
00040     Q_t   q0_;
00041     SetUq Q_;
00042     SetUs sigma_;
00043     Mapt  delta_;
00044     SetUq F_;
00045 };
00046
00047 template <typename Q_t, typename S_t>
00048 DFA<Q_t, S_t>::DFA(const SetUs & sigma, const SetUq & Q,
00049     const SetUq & F,     const Q_t   & q0,
00050     const Mapt  & delta)
00051     : q0_(q0), Q_(Q), sigma_(sigma), delta_(delta), F_(F)
00052 {}
00053
00054 template <typename Q_t, typename S_t>
00055 DFA<Q_t, S_t>::DFA(const DFA & M)
00056     : q0_(M.q0_), Q_(M.Q_), sigma_(M.sigma_), delta_(M.delta_), F_(M.F_)
00057 {}
00058
00059 template <typename Q_t, typename S_t>
00060 bool DFA<Q_t, S_t>::operator()(const vector<S_t> & word, bool verbose)
00061 {
00062     Q_t q = q0_;
00063     std::cout « word « std::endl;
00064
00065     for (int i = 0; i < word.size(); ++i)
00066     {
00067         q = delta_[{q,word[i]}];
00068
00069         if (verbose)
00070         {
```

```
00071             vector<S_t> v(word.begin() + i + 1, word.end());
00072             std::cout « std::pair<Q_t, std::vector<S_t»(q, v) « " " « word[i] « std::endl;
00073         }
00074     }
00075
00076     return is_member(q, F_);
00077 }
00078
00079 template <typename Q_t, typename S_t>
00080 typename DFA<Q_t, S_t>::SuperSetUq
00081 DFA<Q_t, S_t>::Moore_partition_(const SuperSetUq & P) const
00082 {
00083     SuperSetUq next;
00084     for (auto & block: P)
00085     {
00086         SuperSetUq peices = partition(block, P);
00087         next.insert(peices);
00088     }
00089     return next;
00090 }
00091
00092 template <typename Q_t, typename S_t>
00093 typename DFA<Q_t, S_t>::SuperSetUq
00094 DFA<Q_t, S_t>::Moore_partition(const SetUq & S, const SuperSetUq & P)
00095 {
00096     vector<SetUq> vecP(P.begin(), P.end());
00097     int i = 0;
00098
00099     unordered_map< unordered_set<int>, SetUq> P_k;
00100
00101     for (auto q: S)
00102     {
00103         unordered_set<int> index;
00104
00105         for (auto a: sigma_)
00106         {
00107             Q_t qf = delta_[{q, a}];
00108             for (int i = 0; i < vecP.size(); ++i)
00109             {
00110                 if (is_member(qf, vecP[i]))
00111                 {
00112                     index.insert(i);
00113                     break;
00114                 }
00115             }
00116         }
00117         P_k[index].insert(q);
00118     }
00119
00120     SuperSetUq result;
00121     for (auto & kv: P_k)
00122     {
00123         result.insert(kv.second);
00124     }
00125
00126     return result;
00127 }
00128
00129 template <typename Q_t, typename S_t>
00130 DFA<typename DFA<Q_t, S_t>::SetUq, S_t> DFA<Q_t, S_t>::minimizeDFA() const
00131 {
00132     SuperSetUq P;
00133     P.insert(Q_ - F_); P.insert(F_);
00134
00135     while (true)
00136     {
00137         SuperSetUq P_t = Moore_partition(P);
00138
00139         if (P_t == P)
00140             break;
00141
00142         P = P_t;
00143     }
00144
00145     SuperSetUq Q = P;
00146     SetUq q0;
00147
00148     // return DFA< SetUq, S_t>(sigma_, Q, F, q0, delta);
00149 }
00150
00151 template <typename Q_t, typename S_t>
00152 DFA<typename DFA<Q_t, S_t>::PairQQ, S_t>
00153 DFA<Q_t, S_t>::intersection_(const DFA & M) const
00154 {
00155     SetUs       sigma = sigma_ & M.sigma_;
00156     unordered_set<PairQQ> states;
00157     unordered_set<PairQQ> final_ = F_ * M.F_;
```

```
00158     PairQQ      start = PairQQ(q0_, M.q0_);
00159     unordered_map< pair< PairQQ, S_t >, PairQQ > delta;
00160
00161     walk<Q_t, S_t>(start, delta_, M.delta_, delta, sigma, states);
00162
00163     return DFA< PairQQ, S_t >(sigma, states, final_, start, delta);
00164 }
00165
00166 template <typename Q_t, typename S_t>
00167 DFA<typename DFA<Q_t, S_t>::PairQQ, S_t>
00168 DFA<Q_t, S_t>::union_(const DFA & M) const
00169 {
00170     SetUs       sigma = sigma_ & M.sigma_;
00171     unordered_set<PairQQ> states;
00172     unordered_set<PairQQ> final_ = (F_ * M.Q_) | (M.F_ * Q_);
00173     PairQQ      start = PairQQ(q0_, M.q0_);
00174     unordered_map< pair< PairQQ, S_t >, PairQQ > delta;
00175
00176     walk<Q_t, S_t>(start, delta_, M.delta_, delta, sigma, states);
00177
00178     return DFA< PairQQ, S_t >(sigma, states, final_, start, delta);
00179 }
00180
00181 template <typename Q_t, typename S_t>
00182 DFA<Q_t, S_t> & DFA<Q_t, S_t>::compDFA()
00183 {
00184     F_ = Q_ - F_;
00185
00186     return *this;
00187 }
00188
00189 template <typename Q_t, typename S_t>
00190 DFA<int, S_t> baseDFA(int start) const
00191 {
00192     unordered_map<Q_t, int> mapper;
00193     SetUi Q;
00194     Q.insert(start);
00195     mapper[q0_] = start;
00196
00197     SetUq Q_i = Q_;
00198     Q_i.erase(q0_);
00199
00200     SetUi F;
00201     if (is_member(q0_, F_))
00202         F.insert(start);
00203
00204     int i = start + 1;
00205     for (auto p: Q_i)
00206     {
00207         mapper[p] = i;
00208         Q.insert(i);
00209         if (is_member(p, F_))
00210         {
00211             F.insert(i);
00212         }
00213         i++;
00214     }
00215     unordered_map< pair<int, S_t>, int> delta;
00216
00217     for (auto q: Q_)
00218     {
00219         int key = mapper[q];
00220         for (auto a: sigma_)
00221         {
00222             auto it = delta_.find({q, a});
00223             if (it != delta_.end())
00224             {
00225                 auto p = it->second;
00226                 int value = mapper[p];
00227                 delta[{key, a}].insert(value);
00228             }
00229         }
00230     }
00231
00232     return DFA<int, S_t>(sigma_, Q, F, start, delta);
00233 }
00234
00235 template < typename Q_t, typename S_t>
00236 void walk(const pair<Q_t, Q_t> & qi,
00237           const unordered_map< pair<Q_t, S_t>, Q_t> & delta0,
00238           const unordered_map< pair<Q_t, S_t>, Q_t> & delta1,
00239           unordered_map< pair< pair<Q_t, Q_t> , S_t>,
00240           pair<Q_t, Q_t> > & delta_, const unordered_set<S_t> & sigma,
00241           unordered_set< pair<Q_t, Q_t> > & states)
00242 {
00243     if (is_member(qi, states))
00244     {
```

```
00245         return;
00246     }
00247     else
00248     {
00249         states.insert(qi);
00250 
00251         Q_t a = qi.first;
00252         Q_t b = qi.second;
00253 
00254         for (auto p: sigma)
00255         {
00256             pair<Q_t, S_t> key0 =  pair<Q_t, S_t>(a, p);
00257             pair<Q_t, S_t> key1 =  pair<Q_t, S_t>(b, p);
00258 
00259             Q_t value = delta0.at(key0);
00260 
00261             pair<Q_t, Q_t> qi1 = pair<Q_t, Q_t>(delta0.at(key0), delta1.at(key1));
00262             delta_[{qi, p}] = qi1;
00263 
00264             walk(qi1, delta0, delta1, delta_, sigma, states);
00265         }
00266     }
00267 }
00268 
00269 template <typename S_t, typename Q_t >
00270 ostream & operator«(ostream & cout, const DFA< S_t, Q_t > & M)
00271 {
00272     cout « "M:" « endl;
00273     cout « "\tSigma: " « M.sigma_ « endl;
00274     cout « "\tStart State: " « M.q0_ « endl;
00275     cout « "\tSet of States: " « M.Q_ « endl;
00276     cout « "\tTransitions: " « M.delta_ « endl;
00277     cout « "\tFinal States: " « M.F_ « endl;
00278 
00279     return cout;
00280 }
00281 
00282 #endif
```

5.4 include/NFA.h File Reference

NFA implementation.

`#include "STL_util.h"`

Classes

- class NFA< Q_t, S_t >

  Q_t is the type for the for states and the S_t is the type for transitions.

Functions

- template<typename Q_t, typename S_t>
  ostream & **operator**<< (ostream &cout, const NFA< Q_t, S_t > &N)

5.4.1 Detailed Description

NFA implementation.

Author

Oladayo Oyedeji

Date

10/29/2026

NFA is different from a DFA in the sense that the transition is not deterministics hence the name

5.5 NFA.h

Go to the documentation of this file.

```
00001
00010 #ifndef NFA_H
00011 #define NFA_H
00012
00013 #include "STL_util.h"
00018
00019 template <typename Q_t, typename S_t>
00020 class NFA
00021 {
00025     using SetUq = unordered_set<Q_t>;
00026     using SetUs = unordered_set<S_t>;
00027     using SuperSetUq = unordered_set< SetUq >;
00028     using PairQS = pair<Q_t, S_t>;
00029     using PairQQ = pair<Q_t, Q_t>;
00030     using PairS = pair<int, S_t>;
00031     using Mapt = unordered_map<PairQS, SetUq>;
00032     using SetUi = unordered_set<int>;
00033     using Mapi = unordered_map<PairS, SetUi>;
00034     using Map = unordered_map<Q_t, SetUq>;
00035     using MapUi = unordered_map<int, SetUi>;
00036 public:
00037     NFA(const SetUs & sigma, const SetUq & Q,
00038         const SetUq & F,     const Q_t   & q0,
00039         const Mapt  & delta, const Map   & epDt);
00040     NFA(const NFA & N);
00041     NFA()
00042         : q0_(0), Q_({0}), sigma_({}), F_({0})
00043     {}
00044
00045     NFA operator=(const NFA & N)
00046     {
00047         sigma_ = N.sigma_;
00048         Q_     = N.Q_;
00049         F_     = N.F_;
00050         q0_    = N.q0_;
00051         delta_ = N.delta_;
00052         epDt_  = N.epDt_;
00053
00054         return *this;
00055     }
00056     bool operator()(const vector<S_t> & word, bool verbose=false);
00057     NFA<int, S_t> union_(const NFA & N) const;
00058     NFA<int, S_t> concatenate(const NFA & N) const;
00059     NFA<int, S_t> KleneStar() const;
00060     NFA<int, S_t> baseNFA(int start=0) const;
00061     SetUq         epsilon_closure(const SetUq & S) const;
00062     void insert(const S_t & a)
00063     {
00064         int size = Q_.size();
00065         if (size == 0)
00066         {
00067             q0_ = size;
00068         }
00069         Q_.insert(q0_);
00070
00071     }
00072
00073     int size() const
00074     {
00075         return Q_.size();
00076     }
00077
00078     Q_t   q0_;
00079     SetUq Q_;
00080     SetUs sigma_;
00081     Mapt  delta_;
00082     SetUq F_;
00083     Map   epDt_;
00084
00085 };
00086
00087 template <typename Q_t, typename S_t>
00088 NFA<Q_t, S_t>::NFA(const SetUs & sigma, const SetUq & Q,
00089                    const SetUq & F,     const Q_t   & q0,
00090                    const Mapt  & delta, const Map   & epDt)
00091     : q0_(q0), Q_(Q), sigma_(sigma), delta_(delta), F_(F), epDt_(epDt)
00092 {}
00093
00094 template <typename Q_t, typename S_t>
00095 NFA<Q_t, S_t>::NFA(const NFA & N)
00096     : q0_(N.q0_), Q_(N.Q_), sigma_(N.sigma_), delta_(N.delta_), F_(N.F_), epDt_(N.epDt_)
00097 {}
```

```
00098
00099 template <typename Q_t, typename S_t>
00100 bool NFA<Q_t, S_t>::operator()(const vector<S_t> & word, bool verbose)
00101 {
00102     queue< pair<Q_t, int > > fringe;
00103     fringe.push({q0_, 0});
00104     int size = word.size();
00105
00106     SetUq closed_list;
00107     int i = 0;
00108     while (fringe.empty())
00109     {
00110         pair<Q_t, int> top = fringe.pop();
00111         Q_t q = top.first;
00112         int depth = top.second;
00113
00114         closed_list.insert(q);
00115
00116         if (depth >= size)
00117         {
00118             if (is_member(q, F_))
00119                 return true;
00120             continue;
00121         }
00122         auto it = delta_.find({q, word[i]});
00123
00124         if (it != delta_.end())
00125         {
00126             SetUq children_ = it->second;
00127             for (auto qi: children_)
00128             {
00129                 if (!is_member(qi, closed_list))
00130                 {
00131                     fringe.put({qi, depth+1});
00132                 }
00133             }
00134         }
00135         else
00136         {
00137             auto it = epDt_.find(q);
00138             for (auto p: it->second)
00139             {
00140                 if (it != delta_.end())
00141                 {
00142                     if (!is_member(p, closed_list))
00143                         fringe.put({p, depth});
00144                 }
00145             }
00146         }
00147     }
00148     return true;
00149 }
00150
00151 template <typename Q_t, typename S_t>
00152 NFA<int, S_t> NFA<Q_t, S_t>::union_(const NFA & N) const
00153 {
00154     NFA<int, S_t> N0 =   baseNFA(1);
00155     int size_ = size();
00156     NFA<int, S_t> N1 = N.baseNFA(size_ + 1);
00157     cout « N0 « endl;
00158     cout « N1 « endl;
00159     int q0 = 0;
00160     SetUi Q = N0.Q_ | N1.Q_;
00161     Q.insert(q0);
00162
00163     SetUs sigma = N0.sigma_ | N1.sigma_;
00164     Mapi delta = N0.delta_;
00165     delta.merge(N1.delta_);
00166
00167     MapUi epDt = N0.epDt_;
00168     epDt.merge(N1.epDt_);
00169
00170     SetUi F = N0.F_ | N1.F_;
00171
00172     // cout « size_ « ' ' « N1.q0_ « endl;
00173     // cout « N1 « endl;
00174
00175     epDt[q0] = {N0.q0_, N1.q0_};
00176     for (auto a: sigma)
00177     {
00178         // delta[{q0, a}] |= delta_[{  q0_, a}] | N.delta_[{N.q0_, a}];
00179         auto it = N0.delta_.find({N0.q0_, a});
00180         if (it != N0.delta_.end())
00181         {
00182              delta[{q0, a}] = delta[{q0, a}] | it->second;
00183         }
00184         it = N1.delta_.find({N1.q0_, a});
```

```
00185         //cout « it->second « endl;
00186         if (it != N1.delta_.end())
00187         {
00188             // cout « delta[{q0, a}] « ' ' « it->second« endl;
00189             delta[{q0, a}] = delta[{q0, a}] | it->second;
00190         }
00191     }
00192
00193     return NFA<int, S_t>(sigma, Q, F, q0, delta, epDt);
00194 }
00195
00196 template <typename Q_t, typename S_t>
00197 NFA<int, S_t> NFA<Q_t, S_t>::concatenate(const NFA & N) const
00198 {
00199     NFA<int, S_t> N0 =   baseNFA(0);
00200     NFA<int, S_t> N1 = N.baseNFA(size());
00201
00202     cout « "N0: " « N0 « endl;
00203     cout « "N1: " « N1 « endl;
00204
00205     SetUi Q = N0.Q_ | N1.Q_;
00206
00207     SetUs sigma = N0.sigma_ | N1.sigma_;
00208     // cout « sigma « endl;
00209     Mapi delta = N0.delta_;
00210     delta.merge(N1.delta_);
00211
00212     MapUi epDt = N0.epDt_;
00213     epDt.merge(N1.epDt_);
00214
00215     SetUi F = N1.F_;
00216
00217     for (auto q: N0.F_)
00218     {
00219         for (auto a: sigma)
00220         {
00221             // delta[{q,  a}] = delta[{q,a}] | N.delta_[{N.q0_, a}];
00222             auto it = N1.delta_.find({N1.q0_, a});
00223
00224             if (it != N1.delta_.end())
00225             {
00226                 delta[{q, a}] |= it->second;
00227             }
00228         }
00229         epDt[q].insert(N1.q0_);
00230     }
00231
00232     return NFA<int, S_t>(sigma, Q, F, N0.q0_, delta, epDt);
00233 }
00234
00235 template <typename Q_t, typename S_t>
00236 NFA<int, S_t> NFA<Q_t, S_t>::KleneStar() const
00237 {
00238     NFA<int, S_t> N =   baseNFA(1);
00239     cout « N « endl;
00240     int q0 = 0;
00241     SetUi Q = N.Q_;
00242     Q.insert(q0);
00243
00244     SetUs sigma = N.sigma_;
00245     Mapi delta = N.delta_;
00246
00247     MapUi epDt = N.epDt_;
00248     epDt[q0] = {N.q0_};
00249
00250     SetUi F = N.F_;
00251
00252     for (auto q: F)
00253     {
00254         for (auto a: sigma)
00255         {
00256             // delta[{q, a}] = delta[{q, a}] | delta_[{q0_, a}];
00257             auto it = N.delta_.find({N.q0_, a});
00258
00259             if (it != N.delta_.end())
00260             {
00261                 delta[{q, a}] |= it->second;
00262             }
00263         }
00264         epDt[q].insert(q0);
00265     }
00266     cout « F « ' ' « q0 « endl;
00267     F.insert(q0);
00268
00269     return NFA<int, S_t>(sigma, Q, F, q0, delta, epDt);
00270 }
00271
```

```
00272 template <typename Q_t, typename S_t>
00273 NFA<int, S_t> NFA<Q_t, S_t>::baseNFA(int start) const
00274 {
00275     unordered_map<Q_t, int> mapper;
00276     unordered_set<int> Q;
00277     Q.insert(start);
00278     mapper[q0_] = start;
00279 
00280     SetUq Q_i = Q_;
00281     Q_i.erase(q0_);
00282 
00283     unordered_set<int> F;
00284     if (is_member(q0_, F_))
00285         F.insert(start);
00286 
00287     int i = start + 1;
00288     for (auto p: Q_i)
00289     {
00290         mapper[p] = i;
00291         Q.insert(i);
00292         if (is_member(p, F_))
00293         {
00294             F.insert(i);
00295         }
00296         i++;
00297     }
00298 
00299     Mapi delta;
00300     unordered_map<int, unordered_set<int» epDt;
00301 
00302     for (auto q: Q_)
00303     {
00304         int key = mapper[q];
00305         for (auto a: sigma_)
00306         {
00307 
00308             auto it = delta_.find({q, a});
00309             if (it != delta_.end())
00310             {
00311                 for (auto p: it->second)
00312                 {
00313                     int value = mapper[p];
00314                     delta[{key,a}].insert(value);
00315                 }
00316             }
00317         }
00318         auto it = epDt_.find(q);
00319         if (it != epDt_.end())
00320         {
00321             for (auto p: it->second)
00322             {
00323                 int value = mapper[p];
00324                 epDt[key].insert(value);
00325             }
00326         }
00327     }
00328 
00329     return NFA<int, S_t>(sigma_, Q, F, start, delta, epDt);
00330 }
00331 template <typename Q_t, typename S_t >
00332 typename NFA<Q_t, S_t>::SetUq
00333 NFA<Q_t, S_t>::epsilon_closure(const SetUq & S) const
00334 {
00335     SetUq DONE;
00336     SetUq BOUNDARY = S;
00337     SetUq NEW;
00338 
00339     while (true)
00340     {
00341         cout « "oh no" « endl;
00342         for (auto q: BOUNDARY)
00343         {
00344             auto it = epDt_.find(q);
00345             if (it != epDt_.end())
00346             {
00347                 SetUq s = it->second;
00348                 SetUq n = DONE | BOUNDARY;
00349                 NEW = s - n;
00350             }
00351         }
00352         DONE |= BOUNDARY;
00353 
00354         if (NEW.empty())
00355             break;
00356         else
00357         {
00358             BOUNDARY |= NEW;
```

```
00359         }
00360     }
00361
00362     return DONE;
00363 }
00364
00365 template <typename Q_t, typename S_t >
00366 ostream & operator«(ostream & cout, const NFA< Q_t, S_t > & N)
00367 {
00368     cout « "N:" « endl;
00369     cout « "\tSigma: " « N.sigma_ « endl;
00370     cout « "\tStart State: " « N.q0_ « endl;
00371     cout « "\tSet of States: " « N.Q_ « endl;
00372     cout « "\tTransitions: " « N.delta_ « endl;
00373     cout « "\tFinal States: " « N.F_ « endl;
00374     cout « "\tEpsilon transition: " « N.epDt_ « endl;
00375
00376     return cout;
00377 }
00378
00379 #endif
```

5.6 PDA.h

```
00001 #ifndef PDA_H
00002 #define PDA_H
00003
00004 class PDA: public CFG
00005 {
00006     bool operator()(const std::vector<S_t>&);
00007     CYK();
00008     SRP();
00009 };
00010
00011 #endif
```

5.7 Regex.h

```
00001 #ifndef REGEX_H
00002 #define REGEX_H
00003
00004 #include "NFA.h"
00005
00006 typedef unordered_map<int, unordered_set<int» Mapi;
00007 typedef unordered_map<pair<int,char>, unordered_set<int» Mapc;
00008 typedef DFA<unordered_set<int>, char> DFAic;
00009 typedef DFA<int, char> DFAi;
00010 typedef unordered_set<int> Seti;
00011 typedef unordered_set< Seti > SuperSeti;
00012
00013
00014 class RegEx
00015 {
00016 public:
00017     RegEx(char c)
00018         : N_({c}, {0, 1}, {1}, 0,
00019              Mapc{ {0, c}, {1} } },   // 0 --c--> 1
00020              Mapi{}), regex_()
00021     {
00022         // cout « N_ « endl;
00023         regex_ += c;
00024     }
00025     RegEx(const string & x)
00026     {
00027         if (x.size() == 1)
00028         {
00029             *this = RegEx(x[0]);
00030             return;
00031         }
00032         cout « N_ « endl;
00033         for (int i = 0; i < x.size(); ++i)
00034         {
00035             *this &= RegEx(x[i]);
00036         }
00037     }
00038     RegEx(const NFA<int, char> & N)
00039         : N_(N)
00040     {}
```

```
00041     RegEx(const RegEx & r)
00042         : N_(r.N_)
00043     {}
00044     RegEx & operator=(const RegEx & r)
00045     {
00046         N_ = r.N_;
00047         regex_ = r.regex_;
00048 
00049         return *this;
00050     }
00051     // union
00052     RegEx & operator|=(const RegEx & r)
00053     {
00054         N_ = N_.union_(r.N_);
00055         return *this;
00056     }
00057     const RegEx operator|(const RegEx & r) const
00058     {
00059         RegEx R(*this);
00060         R |= r;
00061         return R;
00062     }
00063     // klene star and klene star - {e}
00064     RegEx operator+() const
00065     {
00066         NFA<int, char> N = N_.KleneStar();
00067         N.F_.erase(N_.q0_);
00068 
00069         return RegEx(N);
00070     }
00071     RegEx operator*() const
00072     {
00073         return RegEx(N_.KleneStar());
00074     }
00075     // concatenation
00076     RegEx & operator&=(const RegEx & r)
00077     {
00078         N_ = N_.concatenate(r.N_);
00079         regex_ += r.regex_;
00080         return *this;
00081     }
00082 
00083     RegEx operator&(const RegEx & r) const
00084     {
00085         RegEx R(*this);
00086         R &= r;
00087         return R;
00088     }
00089 
00090     DFAic powerset_construction() const
00091     {
00092         Seti q0 = N_.epsilon_closure({N_.q0_});
00093         int i = 0;
00094         cout « i « " : " « q0 « endl;
00095         // int x;
00096         // cin » x;
00097         SuperSeti Q;
00098         SuperSeti F;
00099         unordered_map<pair<Seti, char>, Seti> delta;
00100 
00101         stack<Seti> fringe;
00102         fringe.push(q0);
00103 
00104         while (!fringe.empty())
00105         {
00106             Seti top = fringe.top();
00107             fringe.pop();
00108             Q.insert(top);
00109 
00110             Seti intersect = top & N_.F_;
00111             F.insert(intersect);
00112 
00113             for (auto a: N_.sigma_)
00114             {
00115                 Seti next;
00116                 for (auto q: top)
00117                 {
00118                     auto it = N_.delta_.find({q, a});
00119                     if (it != N_.delta_.end())
00120                         next |= it->second;
00121                 }
00122 
00123                 next = N_.epsilon_closure(next);
00124                 cout « i « " : " « next « endl;
00125                 // int x;
00126                 // cin » x;
00127                 delta[{top, a}] = next;
```

```
00128
00129                 if (!is_member(next, Q))
00130                 {
00131                     fringe.push(next);
00132                 }
00133             }
00134         }
00135
00136         DFAic M(N_.sigma_, Q, F, q0, delta);
00137
00138         // cout « M.minimizeDFA() « endl;
00139
00140         return M;
00141     }
00142
00143     NFA<int, char> N_;
00144     string regex_;
00145
00146 };
00147
00148
00149 ostream & operator«(ostream & cout, const RegEx & r)
00150 {
00151     cout « "Regular expression: " « r.regex_ « endl;
00152     cout « r.N_;
00153     return cout;
00154 }
00155
00156 #endif
```

5.8 Regular_Language.h

```
00001 #ifndef REGULAR_LANGUAGE_H
00002 #define REGULAR_LANGUAGE_H
00003
00004 #include "DFA.h"
00005 #include "NFA.h"
00006
00007
00008 template <typename Q_t, typename S_t>
00009 std::set<Q_t> epsilon_closure(const std::set<Q_t>& start) const
00010 {
00011     std::set<Q_t> done;
00012     std::set<Q_t> boundary = start;
00013
00014     while (!boundary.empty())
00015     {
00016         std::set<Q_t> next;
00017
00018         for (const auto& q : boundary)
00019         {
00020             done.insert(q);
00021
00022             auto it = delta.find({q, ""});
00023             if (it == delta.end())
00024                 continue;
00025
00026             for (const auto& p : it->second)
00027             {
00028                 if (done.count(p) == 0 && boundary.count(p) == 0)
00029                     next.insert(p);
00030             }
00031         }
00032
00033         boundary = std::move(next);
00034     }
00035
00036     return done;
00037 }
00038
00039 template <typename Q_t, typename S_t>
00040 std::set<Q_t> epsilon_closure(const Q_t& q) const
00041 {
00042     return epsilon_closure(std::set<Q_t>{q});
00043 }
00044
00045 template <typename Q_t, typename S_t>
00046 DFA<std::set<Q_t>> powerset_construction() const
00047 {
00048     using S = std::set<Q_t>;
00049
00050     const S start = epsilon_closure(q0_);
00051
```

```
00052     std::set<S> states;
00053     std::set<S> accepting;
00054     std::map<std::pair<S, std::string>, S> transitions;
00055 
00056     std::stack<S> fringe;
00057     fringe.push(start);
00058     states.insert(start);
00059 
00060     while (!fringe.empty())
00061     {
00062         S top = fringe.top();
00063         fringe.pop();
00064 
00065         // accepting if it contains any NFA accepting state
00066         for (const auto& q : top)
00067         {
00068             if (F_.count(q))
00069             {
00070                 accepting.insert(top);
00071                 break;
00072             }
00073         }
00074 
00075         for (const auto& a : alphabet_)
00076         {
00077             S moved;
00078             for (const auto& q : top)
00079             {
00080                 auto it = delta.find({q, a});
00081                 if (it != delta.end())
00082                     moved.insert(it->second.begin(), it->second.end());
00083             }
00084 
00085             S next = epsilon_closure(moved);
00086             transitions[{top, a}] = next;
00087 
00088             if (states.insert(next).second)   // true if newly discovered
00089                 fringe.push(next);
00090         }
00091     }
00092 
00093     return DFA<S>(states, alphabet_, transitions, start, accepting);
00094 }
00095 
00096 
00097 #endif
```

5.9 STL_util.h

```
00001 #ifndef STL_UTIL_H
00002 #define STL_UTIL_H
00003 
00004 #include <iostream>
00005 #include <vector>
00006 #include <string>
00007 #include <list>
00008 #include <unordered_set>
00009 #include <unordered_map>
00010 #include <set>
00011 #include <stack>
00012 #include <queue>
00013 #include <algorithm>
00014 
00015 using namespace std;
00016 
00017 //============================================================================
00018 // Prototypes
00019 //
00020 // * std::string to_string(T) where T = int, double, char, bool,
00021 // const std::string &
00022 // const std::pair< X, Y > &
00023 // const std::vector< X > &
00024 // const std::list< X > & ... TODO
00025 // const std::unordered_map< X, Y > &
00026 // const std::unordered_set< X > &
00027 //
00028 // * std::hash< T > where T = int, double, char, bool,
00029 // const std::string &
00030 // const std::pair< X, Y > &
00031 // const std::vector< X > &
00032 // const std::list< X > & ... TODO
00033 // const std::unordered_map< X, Y > &
00034 // const std::unordered_set< X > &
```

```
00035 //
00036 // * set membership
00037 // If C is a container and e is an element, use one of the following for
00038 // membership check:
00039 // has< T, S > (C, e)
00040 // is_member< S, T >(e, C) where T is a container of S-values and
00041 // T::find(const S &) is defined
00042 // * set union
00043 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00044 // X |= Y // X = X union with Y
00045 // Z = X | Y // Z = X union with Y
00046 // (See also std::set_union)
00047 //
00048 // * set intersection
00049 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00050 // X &= Y
00051 // Z = X & Y
00052 // (See also C++ std::set_intersection)
00053 //
00054 // * set difference
00055 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00056 // X -= Y
00057 // Z = X - Y
00058 // (See also C++ std::set_difference)
00059 //
00060 // * cross product
00061 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00062 // Z = crossprod(X, Y)
00063 //
00064 // * powerset
00065 // If X, Z are containers of type of std::set< T >, std::unordered_set< T >
00066 // Z = powset(X)
00067 //
00068 //============================================================================
00069
00070 //----------------------------------------------------------------------------
00071 // Prototypes of std::to_string extensions
00072 //----------------------------------------------------------------------------
00073 namespace std
00074 {
00075     std::string to_string(char c);
00076
00077     std::string to_string(const std::string & x);
00078
00079     template < typename S, typename T >
00080     std::string to_string(const std::pair< S, T > & x);
00081
00082     template < typename T >
00083     std::string container_to_string_(const T & x,
00084                                      const std::string & begin,
00085                                      const std::string & end);
00086
00087     template < typename T >
00088     std::string to_string(const std::vector< T > & x);
00089
00090     template < typename T >
00091     std::string to_string(const std::list< T > & x);
00092
00093     template < typename T >
00094     std::string to_string(const std::unordered_set< T > & x);
00095
00096     template < typename S, typename T, typename LESS >
00097     std::string to_string(const std::unordered_map< S, T > & x,
00098                           const LESS & less);
00099
00100     template < typename S, typename T >
00101     std::string to_string(const std::unordered_map< S, T > & x);
00102
00103     template < typename T, typename LESS >
00104     std::string to_string(const std::unordered_set< T > & x,
00105                           const LESS & less);
00106     template < typename T, typename LESS >
00107     std::string to_string(const std::unordered_set< T > & x);
00108
00109     template <typename T >
00110     std::string to_string(const std::set< T > & x);
00111
00112     template < typename T, typename LESS >
00113     std::string to_string(const std::set< T > & x, const LESS & less);
00114 };
00115
00116 //----------------------------------------------------------------------------
00117 // Prototypes of std::hash extensions
00118 //----------------------------------------------------------------------------
00119 namespace std
00120 {
00121     template < typename S, typename T >
```

```
00122     struct hash< std::pair< S, T > >;
00123
00124     template < typename T >
00125     struct hash< std::vector< T > >;
00126
00127     template < typename T >
00128     struct hash< std::list< T > >;
00129
00130     template < typename T >
00131     struct hash< std::unordered_set< T > >;
00132
00133     template < typename S, typename T, typename H >
00134     struct hash< std::unordered_map< S, T, H > >;
00135
00136     template < typename S >
00137     struct hash< std::set< S > >;
00138 }
00139
00140 //--------------------------------------------------------------------------
00141 // Prototypes of operator«
00142 //--------------------------------------------------------------------------
00143 template < typename T0, typename T1 >
00144 std::ostream & operator«(std::ostream & cout, const std::pair< T0, T1 > & x);
00145
00146 template < typename T >
00147 std::ostream & operator«(std::ostream & cout, const std::vector< T > & s);
00148
00149 template < typename T >
00150 std::ostream & operator«(std::ostream & cout, const std::list< T > & s);
00151
00152 template < typename T >
00153 std::ostream & operator«(std::ostream & cout,
00154                          const std::unordered_set< T > & s);
00155
00156 template < typename S, typename T >
00157 std::ostream & operator«(std::ostream & cout,
00158                          const std::unordered_map< S, T > & s);
00159
00160 template < typename T >
00161 std::ostream & operator«(std::ostream & cout, const std::set< T > & s);
00162
00163 //--------------------------------------------------------------------------
00164 // Prototypes of membership
00165 //--------------------------------------------------------------------------
00166 template < typename S, typename T >
00167 bool is_member(const S & e, const T & X);
00168
00169 template < typename S, typename T >
00170 bool has(const S & X, const T & e);
00171
00172 //--------------------------------------------------------------------------
00173 // Prototypes of set union
00174 //--------------------------------------------------------------------------
00175 template < typename T >
00176 void set_union(T & x, const T & y);
00177
00178 template < typename T >
00179 std::set< T > & operator|=(std::set< T > & x, const std::set< T > & y);
00180
00181 template < typename T >
00182 std::set< T > operator|(const std::set< T > & x, const std::set< T > & y);
00183
00184 template < typename T >
00185 std::unordered_set< T > & operator|=(std::unordered_set< T > & x,
00186                                      const std::unordered_set< T > & y);
00187
00188 template < typename T >
00189 std::unordered_set< T > operator|(const std::unordered_set< T > & x,
00190                                   const std::unordered_set< T > & y);
00191
00192 //--------------------------------------------------------------------------
00193 // Prototypes of set intersection
00194 //--------------------------------------------------------------------------
00195 template < typename T >
00196 void set_intersection_(T & xs, const T & ys);
00197
00198 template < typename T >
00199 std::set< T > & operator&=(std::set< T > & xs, const std::set< T > & ys);
00200
00201 template < typename T >
00202 std::set< T > operator&(const std::set< T > & x, const std::set< T > & y);
00203
00204 template < typename T >
00205 std::unordered_set< T > operator&=(std::unordered_set< T > & xs,
00206                                    const std::unordered_set< T > & ys);
00207
00208 template < typename T >
```

```
00209 std::unordered_set< T > operator&(const std::unordered_set< T > & x,
00210                                   const std::unordered_set< T > & y);
00211
00212 //----------------------------------------------------------------------------
00213 // Prototypes for set difference
00214 //----------------------------------------------------------------------------
00215 template < typename T0, typename T1 >
00216 void set_difference_(T0 & X, const T1 & Y);
00217
00218 template < typename T >
00219 const std::unordered_set< T > & operator-=(std::unordered_set< T > & X,
00220                                            const std::unordered_set< T > & Y);
00221
00222 template < typename T >
00223 std::unordered_set< T > operator-(const std::unordered_set< T > & X,
00224                                   const std::unordered_set< T > & Y);
00225
00226 template < typename T >
00227 const std::set< T > & operator-=(std::set< T > & X, const std::set< T > & Y);
00228
00229 template < typename T >
00230 std::set< T > operator-(const std::set< T > & X, const std::set< T > & Y);
00231
00232 //----------------------------------------------------------------------------
00233 // Prototypes of cross product
00234 //----------------------------------------------------------------------------
00235 template < typename T0, typename T1, typename T2 >
00236 void crossprod_(T0 & ret, const T1 & xs, const T2 & ys);
00237
00238 template < typename T0, typename T1 >
00239 std::set< std::pair< T0, T1 > > crossprod(const std::set< T0 > & xs,
00240                                           const std::set< T1 > & ys);
00241
00242 template < typename T0, typename T1 >
00243 std::unordered_set< std::pair< T0, T1 > > crossprod(
00244     const std::unordered_set< T0 > & xs,
00245     const std::unordered_set< T1 > & ys);
00246
00247 template < typename T0, typename T1 >
00248 std::set< std::pair< T0, T1 > > operator*(const std::set< T0 > & xs,
00249                                           const std::set< T1 > & ys);
00250
00251 template < typename T0, typename T1 >
00252 std::unordered_set< std::pair< T0, T1 > > operator*(const std::unordered_set< T0 > & xs,
00253                                           const std::unordered_set< T1 > & ys);
00254
00255 //----------------------------------------------------------------------------
00256 // Prototypes of powerset
00257 //----------------------------------------------------------------------------
00258 template < typename T0, typename T1 >
00259 T0 powset_(const T1 & X);
00260
00261 template < typename T >
00262 std::set< std::set< T > > powset(const std::set< T > & X);
00263
00264 template < typename T >
00265 std::unordered_set< std::unordered_set< T > > powset(
00266     const std::unordered_set< T > & X);
00267
00268 //============================================================================
00269 // Implementations
00270 //============================================================================
00271 //----------------------------------------------------------------------------
00272 // Implementation of std::to_string extensions
00273 //----------------------------------------------------------------------------
00274 namespace std
00275 {
00276     inline
00277     std::string to_string(char c)
00278     {
00279         return {c};
00280     }
00281
00282     inline
00283     std::string to_string(const std::string & x)
00284     {
00285         return x;
00286     }
00287
00288     // template <typename T>
00289     // std::string to_string(const T & x)
00290     // {
00291
00292     // }
00293
00294     template <typename S, typename T>
00295     std::string to_string(const std::pair< S, T > & x)
```

```
00296     {
00297         return std::string("(") +
00298             to_string(x.first) + ", " + to_string(x.second) +
00299             ")";
00300     }
00301
00302     template < typename T >
00303     std::string container_to_string_(const T & x,
00304                                      const std::string & begin,
00305                                      const std::string & end)
00306     {
00307         std::string ret = begin;
00308         std::string delim = "";
00309         for (auto p: x)
00310         {
00311             ret += delim;
00312             ret += to_string(p);
00313             delim = ", ";
00314         }
00315         ret += end;
00316
00317         return ret;
00318     }
00319
00320     template < typename T >
00321     std::string to_string(const std::vector< T > & x)
00322     {
00323         return container_to_string_(x, "[", "]");
00324     }
00325
00326     // template < typename S, typename T, typename LESS >
00327     // std::string to_string(const std::unordered_map< S, T > & x,
00328     //                       const LESS & less)
00329     // {
00330
00331     // }
00332
00333     template < typename S, typename T >
00334     std::string to_string(const std::unordered_map< S, T > & x)
00335     {
00336         return container_to_string_(x, "{", "}");
00337     }
00338
00339     // template < typename T, typename LESS >
00340     // std::string to_string(const std::unordered_set< T > & x,
00341     //                       const LESS & less)
00342     // {
00343     //     return container_to_string_(x, "{", "}");
00344     // }
00345
00346     template < typename T >
00347     std::string to_string(const std::unordered_set< T > & x)
00348     {
00349         return container_to_string_(x, "{", "}");
00350     }
00351
00352     template <typename T >
00353     std::string to_string(const std::set< T > & x)
00354     {
00355         return container_to_string_(x, "{", "}");
00356     }
00357
00358     template < typename T, typename LESS >
00359     std::string to_string(const std::set< T > & x, const LESS & less)
00360     {
00361         return container_to_string_(x, "{", "}");
00362     }
00363 }
00364
00365 //-----------------------------------------------------------------------------
00366 // Implementations of extensions to std::hash
00367 //-----------------------------------------------------------------------------
00368 namespace std
00369 {
00370     template < typename S, typename T >
00371     struct hash< std::pair< S, T > >
00372     {
00373         size_t operator()(const std::pair< S, T > & x) const
00374         {
00375             std::hash< std::string > hasher;
00376             return hasher(std::to_string(x));
00377         }
00378     };
00379
00380     template < typename T >
00381     struct hash< std::vector< T > >
00382     {
```

```
00383         size_t operator()(const std::vector< T > & x) const
00384         {
00385             std::hash< std::string > hasher;
00386             return hasher(std::to_string(x));
00387         }
00388     };
00389
00390     template < typename T >
00391     struct hash< std::list< T > >
00392     {
00393         size_t operator()(const std::list< T > & x) const
00394         {
00395             std::hash< std::string > hasher;
00396             return hasher(std::to_string(x));
00397         }
00398     };
00399
00400     template < typename T >
00401     struct hash< std::unordered_set< T > >
00402     {
00403         size_t operator()(const std::unordered_set< T > & x) const
00404         {
00405             std::hash< std::string > hasher;
00406             return hasher(std::to_string(x));
00407         }
00408     };
00409
00410     template < typename S, typename T, typename H >
00411     struct hash< std::unordered_map< S, T, H > >
00412     {
00413         size_t operator()(const std::unordered_map< S, T, H > & x) const
00414         {
00415             std::hash< std::string > hasher;
00416             return hasher(std::to_string(x));
00417         }
00418     };
00419
00420     template < typename S >
00421     struct hash< std::set< S > >
00422     {
00423         size_t operator()(const std::set< S > & x) const
00424         {
00425             std::hash< std::string > hasher;
00426             return hasher(std::to_string(x));
00427         }
00428     };
00429 }
00430
00431 //--------------------------------------------------------------------------
00432 // Implementation of memberships
00433 //--------------------------------------------------------------------------
00434 template < typename S, typename T >
00435 bool is_member(const S & e,
00436                const T & X)
00437 {
00438     return X.find(e) != X.end();
00439 }
00440
00441 //--------------------------------------------------------------------------
00442 // Implementations of set difference
00443 //--------------------------------------------------------------------------
00444 template < typename T0, typename T1 >
00445 void set_difference_(T0 & X, const T1 & Y)
00446 {
00447     for (auto p: Y)
00448     {
00449         if (is_member(p, X))
00450             X.erase(p);
00451     }
00452 }
00453
00454 template < typename T >
00455 const std::unordered_set< T > & operator-=(std::unordered_set< T > & X,
00456                                            const std::unordered_set< T > & Y)
00457 {
00458     set_difference_(X, Y);
00459     return X;
00460 }
00461
00462 template < typename T >
00463 std::unordered_set< T > operator-(const std::unordered_set< T > & X,
00464                                   const std::unordered_set< T > & Y)
00465 {
00466     unordered_set<T> diff = X;
00467     diff -= Y;
00468     return diff;
00469 }
```

```
00470
00471 template < typename T >
00472 const std::set< T > & operator-=(std::set< T > & X, const std::set< T > & Y)
00473 {
00474     set_difference_(X, Y);
00475     return X;
00476 }
00477
00478 template < typename T >
00479 std::set< T > operator-(const std::set< T > & X, const std::set< T > & Y)
00480 {
00481     std::set<T> diff = X;
00482     diff -= Y;
00483     return diff;
00484 }
00485
00486 //-----------------------------------------------------------------------------
00487 // Implementations of set union
00488 //-----------------------------------------------------------------------------
00489 template < typename T >
00490 void set_union(T & x, const T & y)
00491 {
00492     for (auto p: y)
00493     {
00494         x.insert(p);
00495     }
00496 }
00497 template < typename T >
00498 std::set< T > & operator|=(std::set< T > & x, const std::set< T > & y)
00499 {
00500     set_union(x, y);
00501     return x;
00502 }
00503
00504 template < typename T >
00505 std::set< T > operator|(const std::set< T > & x, const std::set< T > & y)
00506 {
00507     std::set<T> union_ = x;
00508     union_ |= y;
00509     return union_;
00510 }
00511
00512 template < typename T >
00513 std::unordered_set< T > & operator|=(std::unordered_set< T > & x,
00514                                      const std::unordered_set< T > & y)
00515 {
00516     set_union(x, y);
00517     return x;
00518 }
00519
00520 template < typename T >
00521 std::unordered_set< T > operator|(const std::unordered_set< T > & x,
00522                                   const std::unordered_set< T > & y)
00523 {
00524     std::unordered_set<T> union_ = x;
00525     union_ |= y;
00526     return union_;
00527 }
00528
00529 //-----------------------------------------------------------------------------
00530 // Implementations of set intersection
00531 //-----------------------------------------------------------------------------
00532 template < typename T >
00533 void set_intersection_(T & xs, const T & ys)
00534 {
00535     T x = xs;
00536     xs.clear();
00537     for (auto p: x)
00538     {
00539         if (is_member(p, ys))
00540             xs.insert(p);
00541     }
00542 }
00543
00544 template < typename T >
00545 std::set< T > & operator&=(std::set< T > & xs, const std::set< T > & ys)
00546 {
00547     set_intersection_(xs, ys);
00548     return xs;
00549 }
00550
00551 template < typename T >
00552 std::set< T > operator&(const std::set< T > & xs, const std::set< T > & ys)
00553 {
00554     std::set<T> intersect_ = xs;
00555     intersect_ &= ys;
00556     return intersect_;
```

```
00557 }
00558
00559 template < typename T >
00560 std::unordered_set< T > operator&=(std::unordered_set< T > & xs,
00561                                    const std::unordered_set< T > & ys)
00562 {
00563     set_intersection_(xs, ys);
00564     return xs;
00565 }
00566
00567 template < typename T >
00568 std::unordered_set< T > operator&(const std::unordered_set< T > & xs,
00569                                   const std::unordered_set< T > & ys)
00570 {
00571     std::unordered_set<T> intersect_ = xs;
00572     intersect_ &= ys;
00573     return intersect_;
00574 }
00575
00576 //-----------------------------------------------------------------------------
00577 // Implementations of cross product
00578 //-----------------------------------------------------------------------------
00579 template < typename T0, typename T1, typename T2 >
00580 void crossprod_(T0 & ret, const T1 & xs, const T2 & ys)
00581 {
00582     for (auto p: xs)
00583     {
00584         for (auto q: ys)
00585         {
00586             ret.insert(std::make_pair(p, q));
00587         }
00588     }
00589 }
00590
00591 template < typename T0, typename T1 >
00592 std::set< std::pair< T0, T1 > > crossprod(const std::set< T0 > & xs,
00593                                           const std::set< T1 > & ys)
00594 {
00595     std::set< std::pair< T0, T1 > > cP;
00596     crossprod_(cP, xs, ys);
00597
00598     return cP;
00599 }
00600
00601 template < typename T0, typename T1 >
00602 std::unordered_set< std::pair< T0, T1 > > crossprod(
00603     const std::unordered_set< T0 > & xs,
00604     const std::unordered_set< T1 > & ys)
00605 {
00606     std::unordered_set< std::pair< T0, T1 > > cP;
00607     crossprod_(cP, xs, ys);
00608
00609     return cP;
00610 }
00611
00612 template < typename T0, typename T1 >
00613 std::set< std::pair< T0, T1 > > operator*(const std::set< T0 > & xs,
00614                                           const std::set< T1 > & ys)
00615 {
00616     return crossprod(xs, ys);
00617 }
00618
00619 template < typename T0, typename T1 >
00620 std::unordered_set< std::pair< T0, T1 > > operator*(const std::unordered_set< T0 > & xs,
00621                                                     const std::unordered_set< T1 > & ys)
00622 {
00623     return crossprod(xs, ys);
00624 }
00625 //-----------------------------------------------------------------------------
00626 // Implementations of powerset
00627 //-----------------------------------------------------------------------------
00628 template < typename T0, typename T1 >
00629 T0 powset_(const T1 & X)
00630 {
00631     T0 ret;
00632     std::vector<typename T1::value_type> elements(X.begin(), X.end());
00633
00634     int n = elements.size();
00635
00636     for (int mask = 0; mask < (1 << n); ++mask)
00637     {
00638         T1 subset;
00639
00640         for (int i = 0; i < n; ++i)
00641         {
00642             if (mask & (1 << i))
00643                 subset.insert(elements[i]);
```

```
00644         }
00645
00646         ret.insert(subset);
00647     }
00648
00649     return ret;
00650 }
00651
00652 template < typename T >
00653 std::set< std::set< T > > powset(const std::set< T > & X)
00654 {
00655     return powset_(X);
00656 }
00657
00658 template < typename T >
00659 std::unordered_set< std::unordered_set< T > > powset(
00660     const std::unordered_set< T > & X)
00661 {
00662     return powset_(X);
00663 }
00664
00665 //-----------------------------------------------------------------------------
00666 // Implementations of operator«
00667 //-----------------------------------------------------------------------------
00668 template < typename T0, typename T1 >
00669 std::ostream & operator«(std::ostream & cout, const std::pair< T0, T1 > & x)
00670 {
00671     cout « std::to_string(x);
00672     return cout;
00673 }
00674
00675 template < typename T >
00676 std::ostream & operator«(std::ostream & cout, const std::vector< T > & s)
00677 {
00678     cout « to_string(s);
00679     return cout;
00680 }
00681
00682 // template < typename T >
00683 // std::ostream & operator«(std::ostream & cout, const std::list< T > & s)
00684 // {
00685 //     cout « to_string(s);
00686 //     return cout;
00687 // }
00688
00689 template < typename T >
00690 std::ostream & operator«(std::ostream & cout,
00691                          const std::unordered_set< T > & s)
00692 {
00693     cout « to_string(s);
00694     return cout;
00695 }
00696
00697 template < typename S, typename T >
00698 std::ostream & operator«(std::ostream & cout,
00699                          const std::unordered_map< S, T > & s)
00700 {
00701     cout « to_string(s);
00702     return cout;
00703 }
00704
00705 template < typename T >
00706 std::ostream & operator«(std::ostream & cout, const std::set< T > & s)
00707 {
00708     cout « to_string(s);
00709     return cout;
00710 }
00711
00712 #endif
```

5.10 STL_util.h

```
00001 #ifndef STL_UTIL_H
00002 #define STL_UTIL_H
00003
00004 #include <iostream>
00005 #include <vector>
00006 #include <string>
00007 #include <list>
00008 #include <unordered_set>
00009 #include <unordered_map>
00010 #include <set>
00011 #include <algorithm>
```

```
00012
00013 using namespace std;
00014
00015 //============================================================================
00016 // Prototypes
00017 //
00018 // * std::string to_string(T) where T = int, double, char, bool,
00019 // const std::string &
00020 // const std::pair< X, Y > &
00021 // const std::vector< X > &
00022 // const std::list< X > & ... TODO
00023 // const std::unordered_map< X, Y > &
00024 // const std::unordered_set< X > &
00025 //
00026 // * std::hash< T > where T = int, double, char, bool,
00027 // const std::string &
00028 // const std::pair< X, Y > &
00029 // const std::vector< X > &
00030 // const std::list< X > & ... TODO
00031 // const std::unordered_map< X, Y > &
00032 // const std::unordered_set< X > &
00033 //
00034 // * set membership
00035 // If C is a container and e is an element, use one of the following for
00036 // membership check:
00037 // has< T, S > (C, e)
00038 // is_member< S, T >(e, C) where T is a container of S-values and
00039 // T::find(const S &) is defined
00040 // * set union
00041 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00042 // X |= Y // X = X union with Y
00043 // Z = X | Y // Z = X union with Y
00044 // (See also std::set_union)
00045 //
00046 // * set intersection
00047 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00048 // X &= Y
00049 // Z = X & Y
00050 // (See also C++ std::set_intersection)
00051 //
00052 // * set difference
00053 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00054 // X -= Y
00055 // Z = X - Y
00056 // (See also C++ std::set_difference)
00057 //
00058 // * cross product
00059 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00060 // Z = crossprod(X, Y)
00061 //
00062 // * powerset
00063 // If X, Z are containers of type of std::set< T >, std::unordered_set< T >
00064 // Z = powset(X)
00065 //
00066 //============================================================================
00067
00068 //----------------------------------------------------------------------------
00069 // Prototypes of std::to_string extensions
00070 //----------------------------------------------------------------------------
00071 namespace std
00072 {
00073     std::string to_string(char c);
00074
00075     std::string to_string(const std::string & x);
00076
00077     template < typename S, typename T >
00078     std::string to_string(const std::pair< S, T > & x);
00079
00080     template < typename T >
00081     std::string container_to_string_(const T & x,
00082                                      const std::string & begin,
00083                                      const std::string & end);
00084
00085     template < typename T >
00086     std::string to_string(const std::vector< T > & x);
00087
00088     template < typename T >
00089     std::string to_string(const std::list< T > & x);
00090
00091     template < typename T >
00092     std::string to_string(const std::unordered_set< T > & x);
00093
00094     template < typename S, typename T, typename LESS >
00095     std::string to_string(const std::unordered_map< S, T > & x,
00096                           const LESS & less);
00097
00098     template < typename S, typename T >
```

```
00099     std::string to_string(const std::unordered_map< S, T > & x);
00100
00101     template < typename T, typename LESS >
00102     std::string to_string(const std::unordered_set< T > & x,
00103                           const LESS & less);
00104     template < typename T, typename LESS >
00105     std::string to_string(const std::unordered_set< T > & x);
00106
00107     template <typename T >
00108     std::string to_string(const std::set< T > & x);
00109
00110     template < typename T, typename LESS >
00111     std::string to_string(const std::set< T > & x, const LESS & less);
00112 };
00113
00114 //-----------------------------------------------------------------------------
00115 // Prototypes of std::hash extensions
00116 //-----------------------------------------------------------------------------
00117 namespace std
00118 {
00119     template < typename S, typename T >
00120     struct hash< std::pair< S, T > >;
00121
00122     template < typename T >
00123     struct hash< std::vector< T > >;
00124
00125     template < typename T >
00126     struct hash< std::list< T > >;
00127
00128     template < typename T >
00129     struct hash< std::unordered_set< T > >;
00130
00131     template < typename S, typename T, typename H >
00132     struct hash< std::unordered_map< S, T, H > >;
00133
00134     template < typename S >
00135     struct hash< std::set< S > >;
00136 }
00137
00138 //-----------------------------------------------------------------------------
00139 // Prototypes of operator«
00140 //-----------------------------------------------------------------------------
00141 template < typename T0, typename T1 >
00142 std::ostream & operator«(std::ostream & cout, const std::pair< T0, T1 > & x);
00143
00144 template < typename T >
00145 std::ostream & operator«(std::ostream & cout, const std::vector< T > & s);
00146
00147 template < typename T >
00148 std::ostream & operator«(std::ostream & cout, const std::list< T > & s);
00149
00150 template < typename T >
00151 std::ostream & operator«(std::ostream & cout,
00152                          const std::unordered_set< T > & s);
00153
00154 template < typename S, typename T >
00155 std::ostream & operator«(std::ostream & cout,
00156                          const std::unordered_map< S, T > & s);
00157
00158 template < typename T >
00159 std::ostream & operator«(std::ostream & cout, const std::set< T > & s);
00160
00161 //-----------------------------------------------------------------------------
00162 // Prototypes of membership
00163 //-----------------------------------------------------------------------------
00164 template < typename S, typename T >
00165 bool is_member(const S & e, const T & X);
00166
00167 template < typename S, typename T >
00168 bool has(const S & X, const T & e);
00169
00170 //-----------------------------------------------------------------------------
00171 // Prototypes of set union
00172 //-----------------------------------------------------------------------------
00173 template < typename T >
00174 void set_union(T & x, const T & y);
00175
00176 template < typename T >
00177 std::set< T > & operator|=(std::set< T > & x, const std::set< T > & y);
00178
00179 template < typename T >
00180 std::set< T > operator|(const std::set< T > & x, const std::set< T > & y);
00181
00182 template < typename T >
00183 std::unordered_set< T > & operator|=(std::unordered_set< T > & x,
00184                                      const std::unordered_set< T > & y);
00185
```

```
00186 template < typename T >
00187 std::unordered_set< T > operator|(const std::unordered_set< T > & x,
00188                                   const std::unordered_set< T > & y);
00189
00190 //----------------------------------------------------------------------------
00191 // Prototypes of set intersection
00192 //----------------------------------------------------------------------------
00193 template < typename T >
00194 void set_intersection_(T & xs, const T & ys);
00195
00196 template < typename T >
00197 std::set< T > & operator&=(std::set< T > & xs, const std::set< T > & ys);
00198
00199 template < typename T >
00200 std::set< T > operator&(std::set< T > & x, const std::set< T > & y);
00201
00202 template < typename T >
00203 std::unordered_set< T > operator&=(std::unordered_set< T > & xs,
00204                                    const std::unordered_set< T > & ys);
00205
00206 template < typename T >
00207 std::unordered_set< T > operator&(std::unordered_set< T > & x,
00208                                   const std::unordered_set< T > & y);
00209
00210 //----------------------------------------------------------------------------
00211 // Prototypes for set difference
00212 //----------------------------------------------------------------------------
00213 template < typename T0, typename T1 >
00214 void set_difference_(T0 & X, const T1 & Y);
00215
00216 template < typename T >
00217 const std::unordered_set< T > & operator-=(std::unordered_set< T > & X,
00218                                            const std::unordered_set< T > & Y);
00219
00220 template < typename T >
00221 std::unordered_set< T > operator-(const std::unordered_set< T > & X,
00222                                   const std::unordered_set< T > & Y);
00223
00224 template < typename T >
00225 const std::set< T > & operator-=(std::set< T > & X, const std::set< T > & Y);
00226
00227 template < typename T >
00228 std::set< T > operator-(const std::set< T > & X, const std::set< T > & Y);
00229
00230 //----------------------------------------------------------------------------
00231 // Prototypes of cross product
00232 //----------------------------------------------------------------------------
00233 template < typename T0, typename T1, typename T2 >
00234 void crossprod_(T0 & ret, const T1 & xs, const T2 & ys);
00235
00236 template < typename T0, typename T1 >
00237 std::set< std::pair< T0, T1 > > crossprod(const std::set< T0 > & xs,
00238                                           const std::set< T1 > & ys);
00239
00240 template < typename T0, typename T1 >
00241 std::unordered_set< std::pair< T0, T1 > > crossprod(
00242     const std::unordered_set< T0 > & xs,
00243     const std::unordered_set< T1 > & ys);
00244
00245 template < typename T0, typename T1 >
00246 std::set< std::pair< T0, T1 > > operator*(const std::set< T0 > & xs,
00247                                           const std::set< T1 > & ys);
00248
00249 template < typename T0, typename T1 >
00250 std::set< std::pair< T0, T1 > > operator*(const std::unordered_set< T0 > & xs,
00251                                           const std::unordered_set< T1 > & ys);
00252
00253 //----------------------------------------------------------------------------
00254 // Prototypes of powerset
00255 //----------------------------------------------------------------------------
00256 template < typename T0, typename T1 >
00257 T0 powset_(const T1 & X);
00258
00259 template < typename T >
00260 std::set< std::set< T > > powset(const std::set< T > & X);
00261
00262 template < typename T >
00263 std::unordered_set< std::unordered_set< T > > powset(
00264     const std::unordered_set< T > & X);
00265
00266 //============================================================================
00267 // Implementations
00268 //============================================================================
00269 //----------------------------------------------------------------------------
00270 // Implementation of std::to_string extensions
00271 //----------------------------------------------------------------------------
00272 namespace std
```

```
00273 {
00274     inline
00275     std::string to_string(char c)
00276     {
00277         return {c};
00278     }
00279
00280     inline
00281     std::string to_string(const std::string & x)
00282     {
00283         return x;
00284     }
00285
00286     // template <typename T>
00287     // std::string to_string(const T & x)
00288     // {
00289
00290     // }
00291
00292     template <typename S, typename T>
00293     std::string to_string(const std::pair< S, T > & x)
00294     {
00295         return std::string("(") +
00296             to_string(x.first) + ", " + to_string(x.second) +
00297             ")";
00298     }
00299
00300     template < typename T >
00301     std::string container_to_string_(const T & x,
00302                                      const std::string & begin,
00303                                      const std::string & end)
00304     {
00305         std::string ret = begin;
00306         std::string delim = "";
00307         for (auto p: x)
00308         {
00309             ret += delim;
00310             ret += to_string(p);
00311             delim = ", ";
00312         }
00313         ret += end;
00314
00315         return ret;
00316     }
00317
00318     template < typename T >
00319     std::string to_string(const std::vector< T > & x)
00320     {
00321         return container_to_string_(x, "[", "]");
00322     }
00323
00324     // template < typename S, typename T, typename LESS >
00325     // std::string to_string(const std::unordered_map< S, T > & x,
00326     //                       const LESS & less)
00327     // {
00328
00329     // }
00330
00331     template < typename S, typename T >
00332     std::string to_string(const std::unordered_map< S, T > & x)
00333     {
00334         return container_to_string_(x, "{", "}");
00335     }
00336
00337     // template < typename T, typename LESS >
00338     // std::string to_string(const std::unordered_set< T > & x,
00339     //                       const LESS & less)
00340     // {
00341     //     return container_to_string_(x, "{", "}");
00342     // }
00343
00344     template < typename T >
00345     std::string to_string(const std::unordered_set< T > & x)
00346     {
00347         return container_to_string_(x, "{", "}");
00348     }
00349
00350     template <typename T >
00351     std::string to_string(const std::set< T > & x)
00352     {
00353         return container_to_string_(x, "{", "}");
00354     }
00355
00356     template < typename T, typename LESS >
00357     std::string to_string(const std::set< T > & x, const LESS & less)
00358     {
00359         return container_to_string_(x, "{", "}");
```

```
00360     }
00361 }
00362
00363 //-----------------------------------------------------------------------------
00364 // Implementations of extensions to std::hash
00365 //-----------------------------------------------------------------------------
00366 namespace std
00367 {
00368     template < typename S, typename T >
00369     struct hash< std::pair< S, T > >
00370     {
00371         size_t operator()(const std::pair< S, T > & x) const
00372         {
00373             std::hash< std::string > hasher;
00374             return hasher(std::to_string(x));
00375         }
00376     };
00377
00378     template < typename T >
00379     struct hash< std::vector< T > >
00380     {
00381         size_t operator()(const std::vector< T > & x) const
00382         {
00383             std::hash< std::string > hasher;
00384             return hasher(std::to_string(x));
00385         }
00386     };
00387
00388     template < typename T >
00389     struct hash< std::list< T > >
00390     {
00391         size_t operator()(const std::list< T > & x) const
00392         {
00393             std::hash< std::string > hasher;
00394             return hasher(std::to_string(x));
00395         }
00396     };
00397
00398     template < typename T >
00399     struct hash< std::unordered_set< T > >
00400     {
00401         size_t operator()(const std::unordered_set< T > & x) const
00402         {
00403             std::hash< std::string > hasher;
00404             return hasher(std::to_string(x));
00405         }
00406     };
00407
00408     template < typename S, typename T, typename H >
00409     struct hash< std::unordered_map< S, T, H > >
00410     {
00411         size_t operator()(const std::unordered_map< S, T, H > & x) const
00412         {
00413             std::hash< std::string > hasher;
00414             return hasher(std::to_string(x));
00415         }
00416     };
00417
00418     template < typename S >
00419     struct hash< std::set< S > >
00420     {
00421         size_t operator()(const std::set< S > & x) const
00422         {
00423             std::hash< std::string > hasher;
00424             return hasher(std::to_string(x));
00425         }
00426     };
00427 }
00428
00429 //-----------------------------------------------------------------------------
00430 // Implementation of memberships
00431 //-----------------------------------------------------------------------------
00432 template < typename S, typename T >
00433 bool is_member(const S & e,
00434                const T & X)
00435 {
00436     return X.find(e) != X.end();
00437 }
00438
00439 //-----------------------------------------------------------------------------
00440 // Implementations of set difference
00441 //-----------------------------------------------------------------------------
00442 template < typename T0, typename T1 >
00443 void set_difference_(T0 & X, const T1 & Y)
00444 {
00445     for (auto p: Y)
00446     {
```

```
00447         if (is_member(p, X))
00448             X.erase(p);
00449     }
00450 }
00451
00452 template < typename T >
00453 const std::unordered_set< T > & operator-=(std::unordered_set< T > & X,
00454                                            const std::unordered_set< T > & Y)
00455 {
00456     set_difference_(X, Y);
00457     return X;
00458 }
00459
00460 template < typename T >
00461 std::unordered_set< T > operator-(const std::unordered_set< T > & X,
00462                                   const std::unordered_set< T > & Y)
00463 {
00464     unordered_set<T> diff = X;
00465     diff -= Y;
00466     return diff;
00467 }
00468
00469 template < typename T >
00470 const std::set< T > & operator-=(std::set< T > & X, const std::set< T > & Y)
00471 {
00472     set_difference_(X, Y);
00473     return X;
00474 }
00475
00476 template < typename T >
00477 std::set< T > operator-(const std::set< T > & X, const std::set< T > & Y)
00478 {
00479     std::set<T> diff = X;
00480     diff -= Y;
00481     return diff;
00482 }
00483
00484 //-----------------------------------------------------------------------------
00485 // Implementations of set union
00486 //-----------------------------------------------------------------------------
00487 template < typename T >
00488 void set_union(T & x, const T & y)
00489 {
00490     for (auto p: y)
00491     {
00492         x.insert(p);
00493     }
00494 }
00495 template < typename T >
00496 std::set< T > & operator|=(std::set< T > & x, const std::set< T > & y)
00497 {
00498     set_union(x, y);
00499     return x;
00500 }
00501
00502 template < typename T >
00503 std::set< T > operator|(const std::set< T > & x, const std::set< T > & y)
00504 {
00505     std::set<T> union_ = x;
00506     union_ |= y;
00507     return union_;
00508 }
00509
00510 template < typename T >
00511 std::unordered_set< T > & operator|=(std::unordered_set< T > & x,
00512                                      const std::unordered_set< T > & y)
00513 {
00514     set_union(x, y);
00515     return x;
00516 }
00517
00518 template < typename T >
00519 std::unordered_set< T > operator|(const std::unordered_set< T > & x,
00520                                   const std::unordered_set< T > & y)
00521 {
00522     std::set<T> union_ = x;
00523     union_ |= y;
00524     return union_;
00525 }
00526
00527 //-----------------------------------------------------------------------------
00528 // Implementations of set intersection
00529 //-----------------------------------------------------------------------------
00530 template < typename T >
00531 void set_intersection_(T & xs, const T & ys)
00532 {
00533     T x = xs;
```

```
00534     xs.clear();
00535     for (auto p: x)
00536     {
00537         if (is_member(p, ys))
00538             xs.insert(p);
00539     }
00540 }
00541
00542 template < typename T >
00543 std::set< T > & operator&=(std::set< T > & xs, const std::set< T > & ys)
00544 {
00545     set_intersection_(xs, ys);
00546     return xs;
00547 }
00548
00549 template < typename T >
00550 std::set< T > operator&(std::set< T > & xs, const std::set< T > & ys)
00551 {
00552     std::set<T> intersect_ = xs;
00553     intersect_ &= ys;
00554     return intersect_;
00555 }
00556
00557 template < typename T >
00558 std::unordered_set< T > operator&=(std::unordered_set< T > & xs,
00559                                    const std::unordered_set< T > & ys)
00560 {
00561     set_intersection_(xs, ys);
00562     return xs;
00563 }
00564
00565 template < typename T >
00566 std::unordered_set< T > operator&(std::unordered_set< T > & xs,
00567                                   const std::unordered_set< T > & ys)
00568 {
00569     std::unordered_set<T> intersect_ = xs;
00570     intersect_ &= ys;
00571     return intersect_;
00572 }
00573
00574 //-----------------------------------------------------------------------------
00575 // Implementations of cross product
00576 //-----------------------------------------------------------------------------
00577 template < typename T0, typename T1, typename T2 >
00578 void crossprod_(T0 & ret, const T1 & xs, const T2 & ys)
00579 {
00580     for (auto p: xs)
00581     {
00582         for (auto q: ys)
00583         {
00584             ret.insert(std::make_pair(p, q));
00585         }
00586     }
00587 }
00588
00589 template < typename T0, typename T1 >
00590 std::set< std::pair< T0, T1 > > crossprod(const std::set< T0 > & xs,
00591                                           const std::set< T1 > & ys)
00592 {
00593     std::set< std::pair< T0, T1 > > cP;
00594     crossprod_(cP, xs, ys);
00595
00596     return cP;
00597 }
00598
00599 template < typename T0, typename T1 >
00600 std::unordered_set< std::pair< T0, T1 > > crossprod(
00601     const std::unordered_set< T0 > & xs,
00602     const std::unordered_set< T1 > & ys)
00603 {
00604     std::unordered_set< std::pair< T0, T1 > > cP;
00605     crossprod_(cP, xs, ys);
00606
00607     return cP;
00608 }
00609
00610 template < typename T0, typename T1 >
00611 std::set< std::pair< T0, T1 > > operator*(const std::set< T0 > & xs,
00612                                           const std::set< T1 > & ys)
00613 {
00614     return crossprod(xs, ys);
00615 }
00616
00617 template < typename T0, typename T1 >
00618 std::set< std::pair< T0, T1 > > operator*(const std::unordered_set< T0 > & xs,
00619                                           const std::unordered_set< T1 > & ys)
00620 {
```

```
00621     return crossprod(xs, ys);
00622 }
00623 //-----------------------------------------------------------------------------
00624 // Implementations of powerset
00625 //-----------------------------------------------------------------------------
00626 template < typename T0, typename T1 >
00627 T0 powset_(const T1 & X)
00628 {
00629     T0 ret;
00630     std::vector<typename T1::value_type> elements(X.begin(), X.end());
00631
00632     int n = elements.size();
00633
00634     for (int mask = 0; mask < (1 « n); ++mask)
00635     {
00636         T1 subset;
00637
00638         for (int i = 0; i < n; ++i)
00639         {
00640             if (mask & (1 « i))
00641                 subset.insert(elements[i]);
00642         }
00643
00644         ret.insert(subset);
00645     }
00646
00647     return ret;
00648 }
00649
00650 template < typename T >
00651 std::set< std::set< T > > powset(const std::set< T > & X)
00652 {
00653     return powset_(X);
00654 }
00655
00656 template < typename T >
00657 std::unordered_set< std::unordered_set< T > > powset(
00658     const std::unordered_set< T > & X)
00659 {
00660     return powset_(X);
00661 }
00662
00663 //-----------------------------------------------------------------------------
00664 // Implementations of operator«
00665 //-----------------------------------------------------------------------------
00666 template < typename T0, typename T1 >
00667 std::ostream & operator«(std::ostream & cout, const std::pair< T0, T1 > & x)
00668 {
00669     cout « std::to_string(x);
00670     return cout;
00671 }
00672
00673 template < typename T >
00674 std::ostream & operator«(std::ostream & cout, const std::vector< T > & s)
00675 {
00676     cout « to_string(s);
00677     return cout;
00678 }
00679
00680 // template < typename T >
00681 // std::ostream & operator«(std::ostream & cout, const std::list< T > & s)
00682 // {
00683 //     cout « to_string(s);
00684 //     return cout;
00685 // }
00686
00687 template < typename T >
00688 std::ostream & operator«(std::ostream & cout,
00689                          const std::unordered_set< T > & s)
00690 {
00691     cout « to_string(s);
00692     return cout;
00693 }
00694
00695 template < typename S, typename T >
00696 std::ostream & operator«(std::ostream & cout,
00697                          const std::unordered_map< S, T > & s)
00698 {
00699     cout « to_string(s);
00700     return cout;
00701 }
00702
00703 template < typename T >
00704 std::ostream & operator«(std::ostream & cout, const std::set< T > & s)
00705 {
00706     cout « to_string(s);
00707     return cout;
```

```
00708 }
00709
00710 #endif
```

Index

Automata, 7

CFG< V_t, S_t >, 7
CFG< V_t, S_t >::Symbols, 13

DFA< Q_t, S_t >, 8

epsilon, 9

include/Automata.h, 15
include/CFG.h, 15
include/DFA.h, 16
include/NFA.h, 19, 20
include/PDA.h, 24
include/Regex.h, 24
include/Regular_Language.h, 26
include/STL_util.h, 27
include/test/STL_util.h, 35

NFA< Q_t, S_t >, 10

Parse_Tree, 12
Parse_Tree_Node, 12
PDA, 12

RegEx, 13

std::hash< std::list< T > >, 9
std::hash< std::pair< S, T > >, 9
std::hash< std::set< S > >, 9
std::hash< std::unordered_map< S, T, H > >, 10
std::hash< std::unordered_set< T > >, 10
std::hash< std::vector< T > >, 10